!pip install ydata-profiling sweetviz –quiet

This command installs they Y **Data Profiling** and **Sweet viz** libraries, which are powerful tools for automated Exploratory Data Analysis (EDA). These libraries help generate detailed reports containing dataset statistics, missing values, duplicate records, feature distributions, correlations, and data quality insights. The --quiet option suppresses unnecessary installation messages, making the notebook output cleaner and easier to read.

import pandas as pd

import numpy as np

import matplotlib.pyplot as plt

import seaborn as sns

The required Python libraries were imported to perform data analysis and visualization. Pandas was used for data loading, manipulation, and preprocessing. NumPy was used for numerical computations and handling arrays. Matplotlib was used to create basic visualizations, while Seaborn was used to generate advanced statistical plots and graphical representations of the dataset. These libraries provide the foundation for Exploratory Data Analysis (EDA) and Machine Learning workflows.

Pandas → Data manipulation and analysis

NumPy → Numerical operations

Matplotlib → Basic plotting and visualization

Seaborn → Advanced statistical visualizations

Dataset Loading

df = pd.read_csv('customer_churn_dataset.csv')

The dataset was successfully loaded into a Pandas DataFrame. It contains customer demographic information, purchase behavior, membership details, and churn status, which will be used for analysis and prediction.

df.head()

The first five records were examined to understand the dataset structure, column names, and sample values. This helps verify that the data has been loaded correctly.

df.info()

The dataset information shows the total number of records, column names, data types, and non-null values. This helps identify missing values and understand the nature of each feature.

df.describe()

The statistical summary provides information such as mean, median, minimum, maximum, and standard deviation for numerical features. This helps understand the overall distribution of the data.

df.isnull().sum()

Missing values were identified in several columns. These missing values need to be handled before training machine learning models to avoid inaccurate predictions.

df['Age'].fillna(df['Age'].mean(), inplace=True)

Missing numerical values were replaced with the mean value of the respective column. This preserves the overall distribution of the data while handling missing records.

df['Gender'].fillna(df['Gender'].mode()[0], inplace=True)

Missing categorical values were replaced using the most frequent category (mode). This ensures consistency and prevents loss of data.

df.duplicated().sum()

Duplicate records were identified in the dataset. Duplicate entries can bias analysis and model performance, so they must be removed.

df.drop_duplicates(inplace=True)

Duplicate records were removed to improve data quality and ensure that each customer record is unique.

df.fillna(df.mean(numeric_only=True))

Missing values in numerical columns were replaced with their respective mean values. Mean imputation helps preserve the overall distribution of the data while ensuring that no records are lost due to missing values. This step improves data quality and prepares the dataset for further analysis and machine learning model development.

```
import seaborn as sns

import matplotlib.pyplot as plt

plt.figure(figsize=(10,5))

sns.heatmap(df.isnull(), cbar=False)

plt.title("Missing Values Heatmap")

plt.show()
```

A missing values heatmap was generated to visually identify null values present in the dataset. Each collared mark in the heatmap represents a missing value, while blank areas indicate available data. This visualization helps quickly detect columns with missing information and understand the overall pattern of missing data before applying data cleaning techniques.

```
import matplotlib.pyplot as plt

import seaborn as sns

sns.set_style("whitegrid")

plt.figure(figsize=(12,6))

sns.histplot(
```

```
data=df,
    x='Age',
    bins=20,
    kde=True,
    edgecolor='black'
)
plt.title('Customer Age Distribution', fontsize=18)
plt.xlabel('Age')
plt.ylabel('Number of Customers')
plt.tight_layout()
plt.show()
```

This code creates a Histogram with a Kernel Density Estimation (KDE) curve to analyze the distribution of customer ages in the dataset.

A histogram with a density curve was generated to examine the distribution of customer ages. The visualization helps identify the most common age groups, detect potential age-related patterns, and understand the overall demographic structure of customers. The KDE curve provides a smooth representation of the age distribution, making it easier to identify peaks and trends in customer age data.

```
sns.boxplot(
    x=df['Income'],
    color='lightgreen'
)
plt.show()
```

A box plot was used to visualize the distribution of customer income. The plot provides information about the central tendency, spread, and variability of income values. It also helps identify outliers, which are customers whose income is significantly higher or lower than the majority of customers. Understanding income distribution is important because income can influence purchasing behavior and customer churn patterns.

```
sns.barplot(
    x='Gender',
    y='PurchaseAmount',
    data=df,
    color='orange'
) plt.show()
```

The box plot successfully identified the distribution and spread of customer income and highlighted potential outliers. This analysis helps understand customer financial characteristics and supports further machine learning and churn prediction analysis.

```
sns.scatterplot(
    x='Income',
    y='PurchaseAmount',
    hue='Churn',
    data=df
)
plt.show()
```

A scatter plot was used to analyse the relationship between customer income and purchase amount. Different colours were used to represent churned and non-churned customers. The visualization helps identify whether customers with higher incomes tend to spend more and whether spending behavior differs between customers who churn and those who remain with the company.

```
plt.figure(figsize=(10,6))
sns.scatterplot(
    data=df,
    x='Income',
    y='PurchaseAmount',
    hue='CustomerSatisfaction',
    palette='viridis',
    s=100
)
plt.title('Income vs Purchase Amount by Customer Satisfaction')
plt.show()
```

A scatter plot was used to examine the relationship between customer income and purchase amount while incorporating customer satisfaction through colour coding. The visualization helps identify whether customers with higher income tend to spend more and whether highly satisfied customers exhibit different purchasing behaviours compared to less satisfied customers.

```
plt.figure(figsize=(10,6))
sns.heatmap(df.corr(numeric_only=True), annot=True)
plt.show()
```

This heatmap was created to analyse the **correlation between numerical variables** in the customer churn dataset. It helps identify which features are strongly related to each other and which variables may influence customer churn.

```
plt.figure(figsize=(8,5))
sns.countplot(
   x='MembershipType',
   data=df,
   palette='viridis'
)
plt.title('Membership Type Distribution')
plt.show()
```

A correlation heatmap was generated to examine relationships among numerical variables in the dataset. The heatmap visually represents correlation coefficients, allowing identification of features that have strong positive or negative relationships. Understanding these relationships is useful for feature selection, detecting multicollinearity, and identifying factors that may influence customer churn.

```
plt.figure(figsize=(6,4))
sns.countplot(
   x='Churn',
   data=df,
   palette=['#4C72B0', '#DD8452']
)
plt.title('Customer Churn Count')
plt.show()
```

The count plot provided a clear overview of customer retention and churn behavior. It serves as an important first step in understanding the target variable and preparing the dataset for churn prediction modelling.

```
plt.figure(figsize=(8,5))
sns.boxplot(
   x='Churn',
   y='CustomerSatisfaction',
   data=df,
   color='4C72B0'
)
```

```
plt.title('Customer Satisfaction vs Churn', fontsize=14)

plt.show()
```

This box plot was created to compare **Customer Satisfaction** levels between customers who stayed with the company and customers who churned. It helps determine whether customer satisfaction has an impact on customer retention.